SILOSTITCH: Integrating Pretrained Intrusion Detectors under Deployment Budgets

***Abstract*—Organizations may retain intrusion detectors after the records used to train them become unavailable. However, deploying the entire retained pool can exceed a resource budget. Selecting detectors only by average current utility can discard the sole feasible detector that supports an important class, while historical support alone says little about current competence. We present SILOSTITCH, which integrates pretrained detectors under a hard budget without changing them. The method aligns their probability outputs and uses held-out records to form a calibration-eligible pool. It then computes the maximum weighted source-class support exactly and compares current prediction behavior only among sets that preserve this value. Using a disjoint fit partition, the coordinator trains one shared stacker over the selected probability blocks. Clients may add Laplace noise to these blocks before upload. We establish simultaneous post-selection calibration, exact primary optimization with an explicit capacity guard, and a local certificate for secondary refinement. We also prove label-conditional local privacy for score uploads and characterize when current class-conditional utility supports historical class claims under bounded client-mixture change. Across four cybersecurity datasets, a 50% target budget reduces logical payload by 49.3–50.4%. Macro-F1 remains within 0.0052 of all encoders on three datasets; DDOS exhibits a 0.0258 gap.**

I. INTRODUCTION

Operational networks often keep intrusion detectors longer than the records used to train them. Storage and retention policies may remove historical traffic logs [1], [2], even as traffic and attack patterns continue to change [3]. Detectors trained in different organizations may therefore preserve different class support and prediction behavior. Retraining them through another collaborative process would require suitable source data and permission to update the contributed models [4]–[6]. We study post-training integration instead: clients contribute pretrained detectors, and the collaboration leaves their parameters unchanged.

A deployment may still be unable to use every contributed detector. Distributing a model and processing its aligned probability block both consume resources, so the collaboration must select a subset under a hard budget. A utility-only rule may spend that budget on several detectors that repeat common behavior while omitting the sole feasible detector whose source data support a less frequent or high-priority class. Source support alone has the opposite weakness: it can preserve nominal breadth while retaining detectors that perform poorly or add little beyond those already chosen.

One might combine support and current utility in a weighted sum. Yet any fixed coefficient allows enough secondary gain to compensate for a positive support loss when the instance contains a sufficiently small coverage gap. The two quantities therefore play different roles. Source-side class support states what the deployment should retain whenever the budget permits, whereas current behavior distinguishes sets only after that requirement has been met. SILOSTITCH represents this priority as two ordered decisions rather than another scale-dependent tradeoff.

To compare heterogeneous candidates, SILOSTITCH translates every native probability vector into a shared target-class order and reserves one output for unsupported labels. Held-out labeled records then provide normalized Brier utilities, class-conditional lower utilities, and prediction signatures. Separately, fixed source-support counts describe which classes a detector encountered during training, while a logical-byte cost represents model distribution and the probability block used by the shared stacker. These quantities allow the method to distinguish historical support, current behavior, and deployment cost without treating them as interchangeable.

The selector first removes candidates that fail the calibration gate. Within the remaining pool, a sparse bitmask dynamic program obtains the largest weighted source-class support available under the budget. Only sets with that value enter the secondary comparison, which rewards record utility, conservative class utility, and representative prediction behavior. A local fill-and-swap procedure uses any remaining budget without lowering the primary value. Thus, SILOSTITCH combines exact primary optimization with a coverage-preserving fill-and-swap refinement.

Once selection ends, the coordinator trains one shared stacker on a disjoint fit partition using the selected aligned probability blocks [7]. Separate partitions for candidate fitting, selection calibration, stacker fitting, and sealed testing prevent test labels from influencing selection or training. When local score protection is enabled, each client perturbs every selected probability block with Laplace noise before upload. This release provides label-conditional local privacy for feature-derived scores under the label-fixed neighboring relation used in our analysis.

Our analysis follows these dependencies. A simultaneous calibration event keeps detector-level and class-conditional bounds valid after adaptive selection. We then show that the primary problem is NP-hard but solved exactly by the implemented bitmask program within its registered state cap. The secondary objective is monotone and submodular, and the production refinement provides a coverage-preserving one-swap certificate. The Laplace analysis proves local privacy for one uploaded score block and accounts for composition across selected blocks. Finally, an effective-coverage certificate iden-

tifies when current class-conditional utility supports historical class claims, including under bounded changes in the client mixture.

Our evaluation uses four cybersecurity datasets with 66 candidate detectors per task. At a 50% target budget, SILOSTITCH reduces logical payload by 49.3–50.4%; its Macro-F1 remains within 0.0052 of all encoders on MALDROID, DARKNET, and DOH, while DDOS shows a 0.0258 gap. The primary solver matches exhaustive search throughout the oracle suite, and a seven-level Laplace profile quantifies the utility response to score perturbation.

Together, these choices lead to three contributions.

- First, we formulate budgeted post-training detector integration as an ordered decision that separates source-side class support from current predictive evidence.
- Second, we design SILOSTITCH to align frozen detectors, maximize weighted support exactly, refine current behavior without sacrificing that support, and fit one shared stacker from optionally perturbed scores.
- Finally, we establish post-selection calibration, characterize the exact and local parts of the solver, prove label-conditional privacy for score uploads, and analyze effective coverage under client-mixture change.

II. Background and Related Work

This section distinguishes post-training detector integration from collaborative retraining, reviews nearby integration and selection methods, and introduces the privacy notion used for score uploads.

A. Detector Retraining and Adaptation

Federated learning coordinates parameter updates while keeping raw records at their owners, which suits settings in which participants retain training data and can run another optimization process [4]–[6]. Within intrusion detection, federated methods adapt this process to non-IID traffic and heterogeneous local learners [8], [9]. Other recent NIDS methods retrain or adapt detectors to cope with scarce labels, traffic change, or noisy annotations; CoLD, for example, filters unreliable annotations before model fitting [10]–[12]. These methods address important deployment conditions, but all include detector training or adaptation in the collaboration. Our setting begins after the contributed detectors have been trained and their historical records are no longer available.

B. Pretrained Model Integration

When source records disappear but their trained models remain accessible, pretrained-model integration provides the closest starting point. Existing approaches either rank a heterogeneous model zoo on current target data [13] or combine pretrained models without accessing their source records [14], [15]. They do not select one fixed detector subset under a hard aggregate budget while treating class support as a strict priority.

C. Output-Level Ensembles and Cost-Aware Selection

Once model outputs are available, stacked generalization can train a downstream classifier over them [7]. In security, SIRAJ learns a common representation from heterogeneous malicious-entity detectors [16], while Fed-MStacking trains a global meta-classifier from heterogeneous local predictions in a missing-label setting [17]. A separate line studies cost-aware inference: SPIREL learns a per-item policy that trades off model invocations and classification utility [18]. These approaches either combine outputs or control per-item cost, but they do not screen detectors and return one fixed subset under a hard aggregate budget. SILOSTITCH makes this subset decision before fitting one shared classifier over the selected outputs.

D. Local Privacy for Prediction Uploads

Prediction outputs can reveal information about their inputs, so a privacy guarantee must identify both where randomization occurs and which fields it covers. PrivateKT perturbs locally produced predictions before sending them for knowledge transfer [19]. Differentially private federated trees instead protect statistics exchanged while training a new model [20]. Because SILOSTITCH uploads selected detector scores to fit a shared classifier while treating labels as public, we specialize the local model of Duchi et al. [21] to this release.

Definition II.1 (Label-conditional local differential privacy)**.** For a fixed public label y, a client applies a randomized mechanism $\mathcal{M}_y$ to the feature vector x. The upload $\mathcal{R}(x, y) = (\mathcal{M}_y(x), y)$ satisfies label-conditional $(\epsilon, 0)$-local differential privacy if, for every fixed y, every pair x, x', and every measurable output set $\mathcal{O}$,

$$\Pr[\mathcal{M}_y(x) \in \mathcal{O}] \le e^{\epsilon} \Pr[\mathcal{M}_y(x') \in \mathcal{O}]. \tag{1}$$

The definition compares records with the same public label and protects the randomized, feature-derived output.

To instantiate Definition II.1, let a deterministic score map g have replacement sensitivity

$$\Delta_1(g) = \sup_{x,x'} \|g(x) - g(x')\|_1. \tag{2}$$

Adding independent noise $\eta_k \sim \mathrm{Lap}(0, \Delta_1(g)/\epsilon)$ to every coordinate satisfies Definition II.1 [22]. If one record releases blocks with parameters $\epsilon_1, \ldots, \epsilon_s$, sequential composition gives parameter $\sum_{i=1}^{s} \epsilon_i$. Under one-record replacement, releases derived from unchanged records do not add to this privacy parameter.

In SILOSTITCH, this mechanism applies to the selected detector-score blocks. Theorem V.1 establishes label-conditional local privacy for that score upload under the public-label adjacency relation.

E. Capability Comparison

Table I summarizes the closest methods along eight concrete capabilities. The integration columns describe whether a method accepts frozen heterogeneous models and stacks their

outputs. The selection columns describe whether it screens detectors on current data and returns one fixed set under a hard budget while preserving attainable class support as a strict priority. The final column records whether predictions are randomized locally before upload.

A partial mark denotes related but non-equivalent functionality. ZooD's fixed top-K rule is budget-like, but it does not impose an aggregate cost limit. SPIREL incorporates cost into a per-item policy rather than returning one fixed detector set, while PrivateKT aggregates randomized predictions instead of fitting a fixed score stacker. Fed-MStacking addresses missing labels without treating class support as a strict selection priority. These distinctions motivate the ordered selection problem formalized next.

III. Problem Statement

We formalize post-training detector integration, define the ordered selection objective, and derive its selection and privacy guarantees.

A. System Model

After local detector fitting has ended, J clients expose a pool of M candidate detectors. The current implementation assigns one candidate to each client, while the method allows J and M to vary independently. For candidate i, let $p_i(x) \in \mathbb{R}^{h_i}$ denote its native probability output. Because different candidates may use different native label coordinates, the method validates a hard semantic map

$$A_i \in \{0,1\}^{(C+1)\times h_i}, \qquad \mathbf{1}^\top A_i = \mathbf{1}^\top, \tag{3}$$

and forms

$$q_i(x) = A_i p_i(x) \in \Delta^C, \tag{4}$$

where Δ^C is the probability simplex with $C+1$ coordinates. The first C coordinates follow a frozen target ontology, whereas the last coordinate, denoted by $\perp$, collects unsupported outputs. Thus, any mass assigned to $\perp$ remains visible to every later Brier calculation.

Before selection begins, the method freezes every candidate, semantic map, and source-side class count. It also assigns disjoint record partitions to candidate fitting, selection calibration, stacker fitting, and sealed testing. Because test labels remain unopened until both the selected set and stacker have been frozen, they cannot affect calibration, cost, or selection.

The deployment budget uses a logical cost for each selected candidate. Let K_i be candidate i's serialized size, let B_t be the number of stacker-fit records, and let d be the logical byte width of one probability element. We charge for delivering the candidate to J clients and supplying one aligned block for every stacker-fit record:

$$b_i = JK_i + dB_t(C+1). \tag{5}$$

Given a budget fraction β, the registered profile sets

$$B = \left\lfloor \beta \sum_{i=1}^{M} b_i \right\rfloor. \tag{6}$$

The denominator contains every candidate, including those that may later fail calibration. This definition provides a reproducible payload measure for model delivery and aligned score blocks.

B. Detector Selection Objective

With the aligned detector pool and logical budget defined, we next specify what the selector should optimize.

If the coordinator ranks candidates only by average current utility, it may omit the sole contributor that supports an important class. Source-side support alone has the opposite weakness: it can retain candidates whose current probabilities are weak or redundant. A scalar tradeoff can exchange a positive coverage loss for enough secondary gain, so the method gives weighted source-side coverage strict priority and compares current behavior only after that primary value has been fixed.

Accordingly, the intended target is

$$\operatorname{lexmax}_{\varnothing \neq S \subseteq \widehat{\mathcal{R}}} \left(C_{\mathrm{sem}}(S), \Phi(S)\right) \quad \text{s.t.} \quad \sum_{i\in S} b_i \le B. \tag{7}$$

Importantly, Eq. 7 specifies the decision order. The production solver computes its primary coordinate exactly and refines the secondary coordinate heuristically. Only the optional small-pool exhaustive solver computes the complete lexicographic optimum.

C. Execution and Analysis Model

We next specify the execution model and privacy setting used by the analysis.

The current implementation realizes the workflow as one centralized experimental replay. Figure 1 encodes the protocol's logical data dependencies. In a client-side realization, each client perturbs its selected probability blocks with fresh private Laplace randomness before upload. The optional sensitivity mode samples the same distribution centrally to measure utility.

The privacy analysis uses public labels and protects the randomized score blocks used to fit the stacker. It assumes immutable candidate detectors, structurally validated semantic maps, honest source-support reports, and fresh client-side randomness. These assumptions define the cooperative-client execution model analyzed below.

IV. Design of SiloStitch

In this section, we describe how SiloStitch calibrates the frozen candidate pool, selects a budget-feasible detector set, and trains one shared stacker.

A. Selection and Stacking Workflow

Figure 1 separates the records used for detector selection from those used for stacker fitting. Calibration outputs determine eligibility and the selected set, whereas stacker-fit features and score blocks do not enter selection. Clients use them only after that set has been frozen and then upload only the selected aligned blocks, optionally perturbed, and their labels; no contributed detector is updated.

Table I
COMPARISON WITH RECENT RELATED METHODS. HERE, ✓ DENOTES DIRECT SUPPORT, ● RELATED SUPPORT, AND ○ NO SUPPORT.

Method	Model Integration			Subset Selection				Privacy
	Frozen Models	Heterogeneous	Score Stacking	Current Screening	Fixed Set	Hard Budget	Class Priority	Score LDP
SIRAJ [16]	✓	✓	✓	○	○	○	○	○
PrivateKT [19]	○	○	●	○	○	○	○	✓
SPIREL [18]	✓	✓	●	○	●	●	○	○
ZooD [13]	✓	✓	●	✓	✓	●	○	○
Fed-MStacking [17]	○	✓	✓	○	○	○	○	○
SILOSTITCH	✓	✓	✓	✓	✓	✓	✓	✓

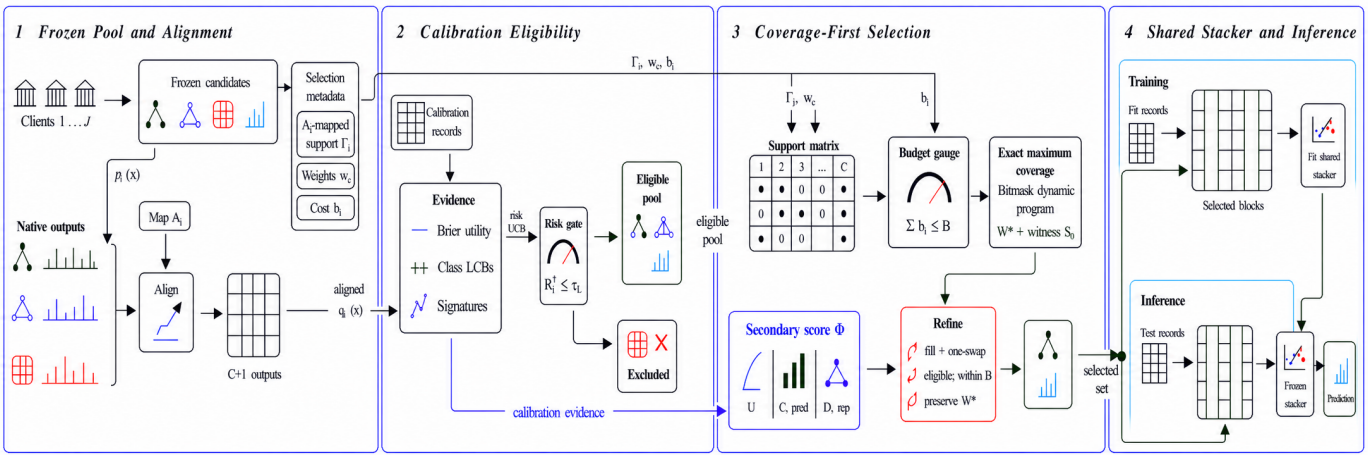

Figure 1. SILOSTITCH first freezes and aligns the candidate detector pool. It then uses selection-calibration records to form the eligible pool, obtains the exact maximum weighted semantic coverage under the logical budget, and performs coverage-preserving local secondary refinement. Finally, selected aligned probability blocks train one shared stacker and support inference. Arrows encode the protocol's logical data dependencies.

Algorithm 1 SILOSTITCH Client–Server Procedure

Require: $\{f_i, A_i, H_i, K_i\}_{i=1}^{M}$, $\mathcal{D}^{\mathrm{cal}}$, $\{\mathcal{D}_j^{\mathrm{stk}}\}_{j=1}^{J}$, B, $\epsilon_b \in (0, \infty]$
Ensure: $\widehat{S}$, $\widehat{f}$
Clients $\to$ Server: $\{f_i, A_i, H_i, K_i\}_{i=1}^{M}$
$(\widehat{\mathcal{R}}, \Gamma, w, b, \Phi) \leftarrow \textsc{Calibrate}(\mathcal{D}^{\mathrm{cal}})$
$(W^\star, S_0) \leftarrow \textsc{Cover}(\widehat{\mathcal{R}}, \Gamma, w, b, B)$
$\widehat{S} \leftarrow \textsc{Refine}(S_0, W^\star, \Phi, B)$
Server $\to$ Clients: $\{f_i, A_i\}_{i \in \widehat{S}}$
for $j = 1, \ldots, J$ **in parallel do**
  $Z_j \leftarrow \mathrm{concat}_{i \in \widehat{S}}\, q_i(X_j^{\mathrm{stk}})$
  $(\Xi_j)_{t,i,k} \overset{\mathrm{iid}}{\sim} \mathrm{Lap}(0, 2/\epsilon_b)$
  $\widetilde{Z}_j \leftarrow Z_j + \Xi_j$
  Client $j \to$ Server: $(\widetilde{Z}_j, Y_j^{\mathrm{stk}})$
end for
$\widehat{f} \leftarrow \textsc{Fit}(\bigcup_j (\widetilde{Z}_j, Y_j^{\mathrm{stk}}))$
return $(\widehat{S}, \widehat{f})$

With these stages in place, Algorithm 1 summarizes the complete client–server procedure, while the following subsections define CALIBRATE, COVER, and REFINE in detail. Here, ϵ_b is the privacy parameter assigned to one selected detector block; $\epsilon_b = \infty$ disables perturbation, so $\mathrm{Lap}(0, 0)$ denotes zero noise.

B. Probability Alignment and Brier Calibration

Following the workflow overview, we first specify how aligned outputs are calibrated and summarized.

For calibration record (x_t, y_t), let $e_{y_t} \in \mathbb{R}^{C+1}$ be the one-hot label vector with a zero $\perp$ coordinate. Because the implementation uses the complete aligned output, it defines

$$\ell_{i,t} = \frac{1}{2} \left\| q_i(x_t) - e_{y_t} \right\|_2^2, \qquad g_{i,t} = 1 - \ell_{i,t} \in [0, 1]. \tag{8}$$

Thus, $g_{i,t}$ is one full-vector Brier utility rather than a collection of coordinate-wise scores.

Eligibility screening. To screen out candidates whose average current loss is either too large or too uncertain, the method uses N calibration records and failure level $\delta \in (0, 1)$ to compute

$$\widehat{\mu}_i = \frac{1}{N} \sum_{t=1}^{N} g_{i,t}, \qquad r = \sqrt{\frac{\log(4M/\delta)}{2N}}, \tag{9}$$

and admits candidate i only if

$$R_i^+ = 1 - \widehat{\mu}_i + r \le \tau_L. \tag{10}$$

Accordingly, the calibration-eligible pool is

$$\widehat{\mathcal{R}} = \{ i : R_i^+ \le \tau_L \}. \tag{11}$$

Thus, eligibility depends on an upper confidence bound on risk rather than on empirical utility alone.

Classwise evidence. Average eligibility does not preserve class-specific evidence for later refinement and certification. For class c, let $N_c = \sum_t \mathbf{1}\{y_t = c\}$ and

$$\widehat{\mu}_{i,c} = \frac{1}{N_c} \sum_{t:y_t=c} g_{i,t}. \tag{12}$$

Because the protocol requires every calibration class to occur, $N_c > 0$. The class-conditional radius and lower utility are

$$r_c = \sqrt{\frac{\log(4MC/\delta)}{2N_c}}, \qquad a_{i,c} = [\widehat{\mu}_{i,c} - r_c]_+, \tag{13}$$

where $[z]_+ = \max\{0, z\}$. Hence, $a_{i,c}$ is the class-specific lower score used by the secondary objective and the effective-coverage certificate.

Behavioral similarity. Two eligible candidates can still behave almost identically. To measure this redundancy, the method deterministically selects a class-stratified anchor set $\mathcal{A}$ and forms

$$z_i = \mathrm{vec}\left([q_i(x_t)]_{t\in\mathcal{A}}\right), \qquad \kappa_{i,k} = \left[\frac{\langle z_i, z_k\rangle}{\|z_i\|_2 \|z_k\|_2}\right]_0^1. \tag{14}$$

Here, $[\cdot]_0^1$ clips numerical roundoff into $[0, 1]$. We use $\kappa_{i,k}$ in a facility-location term that measures representativeness, not as a pairwise-diversity score.

C. Coverage-Prioritized Detector Selection

Calibration now supplies the evidence needed for budgeted selection. We first define the primary support objective.

Weighted semantic coverage. For candidate i and target class c, let $H_{i,c}$ be the accepted source-side support count after semantic mapping. After fixing $n_{\min} > 0$, define

$$\Gamma_{i,c} = \mathbf{1}\{H_{i,c} \geq n_{\min}\}. \tag{15}$$

If $H_c = \sum_i H_{i,c} > 0$, the registered profile uses

$$w_c = \frac{H_c^{-1/2}}{\sum_{d=1}^{C} H_d^{-1/2}}, \qquad \sum_{c=1}^{C} w_c = 1. \tag{16}$$

Thus, classes with less aggregate source support receive greater weight. Then, for any selected set S,

$$C_{\mathrm{sem}}(S) = \sum_{c=1}^{C} w_c \mathbf{1}\{\exists i \in S : \Gamma_{i,c} = 1\}. \tag{17}$$

The $\perp$ coordinate never contributes to this coverage. Consequently, C_{sem} measures the breadth of declared source support rather than current predictive accuracy.

Secondary behavioral objective. Only after fixing semantic coverage does the method compare current predictive behavior. With $h_\tau(z) = 1 - e^{-z/\tau}$, define

$$\begin{aligned} U(S) &= \frac{1}{N} \sum_{t=1}^{N} h_{\tau_U}\left(\sum_{i\in S} g_{i,t}\right), \\ C_{\mathrm{pred}}(S) &= \frac{1}{C} \sum_{c=1}^{C} h_{\tau_C}\left(\sum_{i\in S} a_{i,c}\right), \\ D_{\mathrm{rep}}(S) &= \frac{1}{|\widehat{\mathcal{R}}|} \sum_{k\in\widehat{\mathcal{R}}} \max_{i\in S} \kappa_{i,k}. \end{aligned} \tag{18}$$

Here, U summarizes record-level utility, C_{pred} summarizes conservative classwise evidence, and D_{rep} rewards behavior that represents the eligible pool. Here and throughout the analysis, we set $\max_{i\in\varnothing} \kappa_{i,k} = 0$, matching the implementation's zero-valued empty-set convention. Unlike C_{sem}, C_{pred} averages classes uniformly, exactly as in the implementation. After fixing nonnegative weights that sum to one, the secondary objective is

$$\Phi(S) = \lambda_U U(S) + \lambda_C C_{\mathrm{pred}}(S) + \lambda_D D_{\mathrm{rep}}(S). \tag{19}$$

This score is used only after semantic coverage has been fixed.

Budgeted selection procedure. We therefore define the feasible family and its optimal primary value as

$$\begin{aligned} \mathcal{F}_B &= \left\{\varnothing \neq S \subseteq \widehat{\mathcal{R}} : \sum_{i\in S} b_i \leq B\right\}, \\ W^\star &= \max_{S\in\mathcal{F}_B} C_{\mathrm{sem}}(S). \end{aligned} \tag{20}$$

To compute $W^\star$, the production solver encodes every Γ_i as a C-bit mask and uses a sparse dynamic program to recover a least-cost witness. It then spends any remaining budget through density-based fill and accepts a one-for-one replacement only when the change preserves $W^\star$ and improves Φ by more than the registered tolerance $\varepsilon_{\mathrm{loc}}$.

For a matched comparison, the implementation also applies the same fill–one-swap procedure without semantic priority. If that no-semantic result already attains $W^\star$, SILOSTITCH returns the same set unchanged. Otherwise, it refines the exact coverage witness and, when available, a coverage-optimal singleton seed, then returns the better locally refined result. Thus, when the semantic constraint is inactive, both variants return the same set; they differ only when semantic priority changes the refinement path.

D. Shared Stacker Learning

After the selector fixes $\widehat{S}$, the selected representation is

$$\phi_{\widehat{S}}(x) = \mathrm{concat}_{i\in\widehat{S}}\, q_i(x) \in \mathbb{R}^{|\widehat{S}|(C+1)}. \tag{21}$$

Local score perturbation. When local score protection is enabled, client j independently perturbs every coordinate of each selected block:

$$\widetilde{q}_i(x_t) = q_i(x_t) + \eta_{i,t}, \qquad \eta_{i,t,k} \overset{\mathrm{iid}}{\sim} \mathrm{Lap}\left(0, \frac{2}{\epsilon_b}\right). \tag{22}$$

We write the perturbed representation as

$$\widetilde{\phi}_{\widehat{S}}(x) = \mathrm{concat}_{i\in\widehat{S}}\, \widetilde{q}_i(x). \tag{23}$$

Shared stacker fitting. Using the independent stacker-fit partition $\mathcal{D}^{\mathrm{stk}}$, the coordinator then trains

$$\widehat{f}_{\widehat{S}} = \mathsf{Fit}_{\mathrm{global}}\left(\{(\widetilde{\phi}_{\widehat{S}}(x_t), y_t) : (x_t, y_t) \in \mathcal{D}^{\mathrm{stk}}\}\right). \tag{24}$$

For the nonprivate configuration, the method sets $\widetilde{\phi}_{\widehat{S}} = \phi_{\widehat{S}}$. Although the implementation may cache a full aligned block matrix for efficiency, both configurations use only the columns indexed by $\widehat{S}$. Unselected blocks therefore cannot affect the learned classifier.

The optional centralized sensitivity mode samples Eq. 22 after selection to measure the effect of noise. A distributed realization samples the same noise privately at each client. At sealed-test inference, the method uses the same selected layout and leaves every contributed detector unchanged.

V. Theoretical Analysis

In this section, we establish guarantees for score-upload privacy, calibration, primary optimization, and local secondary refinement. We then analyze how client-mixture shift affects the interpretation of current evidence.

A. Local Privacy of Score Uploads

We first analyze the privacy of the score blocks randomized before upload.

We fix the selected set before observing any stacker-fit record and treat each record label as public. Accordingly, (x, y) and (x', y) are adjacent only when their feature vectors differ. This label-fixed relation is necessary because Algorithm 1 uploads y without perturbation. For a batched upload, adjacent batches have the same size, row alignment, and label sequence, and they differ in exactly one feature vector. Each client samples noise independently across records, selected detector blocks, and output coordinates.

Theorem V.1 (Label-conditional local score privacy)**.** Suppose that the selected detectors and their semantic maps are fixed and that $0 < \epsilon_b < \infty$ and each client uses fresh private randomness in Eq. 22. Then one released detector block is $(\epsilon_b, 0)$-locally differentially private under the preceding adjacency relation. If $s = |\widehat{S}|$ blocks are released for the same record, their joint release is $(s\epsilon_b, 0)$-locally differentially private. Under one-record replacement, releasing the entire client batch has the same record-level guarantee. More generally, assigning parameter ϵ_i to block i gives $(\sum_{i\in\widehat{S}} \epsilon_i, 0)$-local differential privacy.

Proof. Because $q_i(x)$ and $q_i(x')$ are probability vectors, $\|q_i(x) - q_i(x')\|_1 \le 2$. Therefore, coordinatewise Laplace noise with scale $2/\epsilon_b$ bounds the density ratio of one released block by e^{ϵ_b}. Sequential composition over the s selected blocks gives $e^{s\epsilon_b}$. Because different rows use independent randomization and neighboring batches differ in one row, parallel composition across rows leaves this bound unchanged. Appending the unchanged public labels does not alter the density ratio. □

Thus, a total per-record score budget ϵ can be obtained by assigning $\epsilon_b = \epsilon/s$ to each block, which changes the noise scale to $2s/\epsilon$. This is a label-conditional, record-level guarantee for the randomized score upload. Repeated uploads compose across rounds.

B. Calibration under Heterogeneous Clients

The privacy result concerns the fit partition, whereas detector eligibility depends on separate calibration records. We therefore analyze those statistics next.

Because calibration records can originate from different clients and classes, we condition on both the observed client–label design $\mathcal{I} = \{(j_t, y_t)\}_{t=1}^N$ and the candidate pool $\mathcal{Q}$ frozen by the preceding fit stage. We assume that calibration records are conditionally independent given $(\mathcal{Q}, \mathcal{I})$, although their conditional distributions may differ across t. Define

$$\mu_i = \frac{1}{N}\sum_{t=1}^{N} \mathbb{E}[g_{i,t} \mid \mathcal{Q}, \mathcal{I}], \qquad R_i = 1 - \mu_i, \tag{25}$$

and

$$\mu_{i,c} = \frac{1}{N_c}\sum_{t:y_t=c} \mathbb{E}[g_{i,t} \mid \mathcal{Q}, \mathcal{I}]. \tag{26}$$

Thus, these quantities describe the record-weighted calibration mixture rather than equal-client or worst-client performance.

Theorem V.2 (Simultaneous calibration)**.** Under the preceding assumptions, with probability at least $1 - \delta$, the following inequalities hold simultaneously for every candidate i and class c:

$$R_i \le R_i^+ \le R_i + 2r, \tag{27}$$

$$\max\{0, \mu_{i,c} - 2r_c\} \le a_{i,c} \le \mu_{i,c}. \tag{28}$$

Consequently, the inequalities remain valid for every candidate in any set selected adaptively from the calibration statistics.

Proof. Conditioning on $(\mathcal{Q}, \mathcal{I})$, Hoeffding's inequality gives

$$\Pr\{|\widehat{\mu}_i - \mu_i| > r \mid \mathcal{Q}, \mathcal{I}\} \le 2e^{-2Nr^2} = \frac{\delta}{2M}.$$

Therefore, a union bound over M candidates spends at most $\delta/2$. Likewise,

$$\Pr\{|\widehat{\mu}_{i,c} - \mu_{i,c}| > r_c \mid \mathcal{Q}, \mathcal{I}\} \le 2e^{-2N_c r_c^2} = \frac{\delta}{2MC},$$

so a second union bound over MC pairs spends at most $\delta/2$. The intersection has probability at least $1 - \delta$, and it does not require different candidates evaluated on the same record to be independent.

On this event, $1 - \widehat{\mu}_i + r$ lies between R_i and $R_i + 2r$. Moreover, $\widehat{\mu}_{i,c} - r_c$ lies between $\mu_{i,c} - 2r_c$ and $\mu_{i,c}$. Applying $[\cdot]_+$ proves the class inequalities. Because the event covers all $M + MC$ coordinates before selection, taking a data-dependent subset requires no additional union bound over 2^M sets. □

For any pool $\mathcal{E}$, let $W(\mathcal{E})$ be the largest feasible semantic coverage available from that pool, with value zero if no nonempty set is feasible. On the event in Theorem V.2, define

$$\mathcal{R}^- = \{i : R_i \le \tau_L - 2r\}, \qquad \mathcal{R}^0 = \{i : R_i \le \tau_L\}. \quad (29)$$

Then,

$$\mathcal{R}^- \subseteq \widehat{\mathcal{R}} \subseteq \mathcal{R}^0, \qquad W(\mathcal{R}^-) \le W(\widehat{\mathcal{R}}) \le W(\mathcal{R}^0). \quad (30)$$

Indeed, the left inclusion follows from $R_i^+ \le R_i + 2r$, while the right inclusion follows from $R_i \le R_i^+$. Thus, if one primary-optimal witness over $\mathcal{R}^0$ lies entirely in $\mathcal{R}^-$, calibration preserves the population-mixture primary value.

With the registered profile's $\tau_L = 1$, the upper-confidence gate acts as a finite-sample admissibility screen over normalized Brier risk. We therefore refer to $\widehat{\mathcal{R}}$ as the calibration-eligible pool.

C. Exact Primary Coverage under a Hard Budget

The simultaneous event validates the statistics used to form the eligible pool. With that pool fixed, we now characterize the primary selection problem and its exact solver.

Theorem V.3 (Primary hardness and exact parameterized solver)**.** Computing $W^\star$ is NP-hard, even when every candidate is eligible, every cost is one, and all class weights are uniform. Nevertheless, if the registered state cap is not exceeded, the bitmask dynamic program returns a nonempty feasible set S_0 with

$$C_{\mathrm{sem}}(S_0) = W^\star. \quad (31)$$

It stores at most 2^C masks and examines at most $|\widehat{\mathcal{R}}|2^C$ state transitions. Thus, its primary stage is fixed-parameter tractable in C.

Proof. For hardness, take a Maximum Coverage instance with universe $\{1, \dots, C\}$, subsets $\Gamma_1, \dots, \Gamma_M$, and cardinality limit k. Set $b_i = 1$, $B = k$, and $w_c = 1/C$. Maximizing C_{sem} then solves that instance up to the constant factor $1/C$.

For exactness, after processing the first t eligible candidates, associate each reachable union mask h with its least-cost witness. Omitting candidate t preserves an earlier state, whereas including it extends an earlier mask to $h \vee \Gamma_t$. Every subset makes exactly one of these choices. Moreover, a lower-cost witness for the same mask dominates every higher-cost witness under all future union operations. Induction therefore shows that the final table contains the least-cost witness for every reachable mask. Scanning the budget-feasible masks returns exactly $W^\star$.

A C-bit mask has at most 2^C values, and each eligible candidate extends each current mask at most once. When all attainable masks have value zero, the implementation returns the least-cost feasible singleton, which enforces the nonempty stacker input without changing the primary value. □

The current Python implementation additionally stores, copies, and sorts complete witness tuples, so its wall-clock time includes work beyond the state transition count. A requirement above 10^6 states triggers an explicit capacity error, preserving the exact-solver contract.

D. Secondary Objective and Local Refinement

The primary solver determines $W^\star$. The remaining question is how the method compares feasible sets that attain this value.

Let $\mathcal{V} = \{C_{\mathrm{sem}}(S) : S \in \mathcal{F}_B\}$ contain the attainable primary values. When $\mathcal{V}$ has at least two values, let Δ_{inst} be its smallest positive gap. Because $0 \le \Phi(S) \le 1$, any coefficient $K > 1/\Delta_{\mathrm{inst}}$ is sufficient to make every maximizer of $KC_{\mathrm{sem}} + \Phi$ lexicographic for that fixed instance. Normalized class weights can make the smallest positive coverage gap arbitrarily small, so a universal finite coefficient is unavailable. The strict order in Eq. 7 does not require instance-specific scale tuning.

Proposition V.4 (Secondary structure)**.** For fixed calibration statistics, U, C_{pred}, and D_{rep} are normalized, monotone, submodular set functions. Consequently, their nonnegative weighted combination Φ has the same properties.

Proof. For U and C_{pred}, each summand applies the nondecreasing concave function h_τ to a nonnegative modular sum. Therefore, the gain from adding the same candidate cannot increase as the current set grows. For D_{rep}, target k contributes $\max_{i \in S} \kappa_{i,k}$, whose marginal gain is

$$\max\{0, \kappa_{a,k} - \max_{i \in S} \kappa_{i,k}\}.$$

This gain also cannot increase as S grows. Averaging and taking a nonnegative weighted sum preserve all three properties. □

Although Φ is monotone submodular, the equal-primary family

$$\mathcal{F}^\star = \{S \in \mathcal{F}_B : C_{\mathrm{sem}}(S) = W^\star\} \quad (32)$$

is not downward closed and need not satisfy an exchange axiom. Therefore, standard monotone-submodular knapsack guarantees, including $1 - 1/e$, do not apply to the production secondary stage.

For $S \in \mathcal{F}^\star$, let $\mathcal{N}_1(S)$ contain every set obtained by removing exactly one selected candidate and adding exactly one unselected candidate while preserving budget feasibility and $W^\star$.

Theorem V.5 (Coverage preservation and one-swap certificate)**.** If the primary dynamic program returns normally, the production refinement terminates after finitely many accepted operations and returns $\widehat{S} \in \mathcal{F}^\star$ such that

$$\Phi(S') \le \Phi(\widehat{S}) + \varepsilon_{\mathrm{loc}}, \qquad \forall S' \in \mathcal{N}_1(\widehat{S}). \quad (33)$$

Proof. The constrained seed has coverage $W^\star$. Because C_{sem} is monotone and $W^\star$ is already maximal, every feasible fill keeps that value. The implementation separately checks every accepted replacement for budget feasibility and equality to $W^\star$. Each accepted operation also increases Φ by more than $\varepsilon_{\mathrm{loc}}$. Because the candidate pool has only finitely many subsets, the procedure terminates. At termination, the exhaustive

one-for-one search has found no feasible coverage-preserving replacement whose gain exceeds the tolerance, which proves Eq. 33. □

The certificate covers the production neighborhood of coverage-preserving one-for-one replacements. The fill stage ranks candidates by gain per byte and stops when the chosen candidate's absolute gain falls below tolerance; broader additions and multi-candidate exchanges remain separate optimization choices.

E. Effective Coverage under Client Mixture Shift

The preceding results treat declared support and current predictive behavior as separate quantities. We now ask when the former is backed by the latter and how that relation changes under client-mixture shift.

We use the following quantities as post-selection certificates for the frozen returned set, reusing statistics already computed during calibration.

To test whether declared source support is backed by current evidence, let $\vartheta_c \in [0,1]$ be a prespecified current-competence threshold and define

$$C_{\mathrm{cert}}(S;\boldsymbol{\vartheta}) = \sum_{c=1}^{C} w_c \mathbf{1}\{\exists i \in S : \Gamma_{i,c} = 1 \wedge a_{i,c} \geq \vartheta_c\}. \quad (34)$$

Thus, a class contributes only when a selected detector provides both declared source support and a conservative current-utility score. For comparison, let $C_\mu^a(S;\boldsymbol{\vartheta})$ replace $a_{i,c} \geq \vartheta_c$ by $\mu_{i,c}^a \geq \vartheta_c$, where $a \in \{\mathrm{cal}, \mathrm{dep}\}$.

Let $h = (j,c)$ index a client–class cell. Suppose that detector behavior within each cell remains stable between calibration and deployment, while only the mixture weights change. Let $R_{i,h} \in [0,1]$ denote the cell risk, and let π_h^{cal} and π_h^{dep} denote the two mixtures. For $a \in \{\mathrm{cal}, \mathrm{dep}\}$, define

$$R_i^a = \sum_{j,c} \pi_{j,c}^a R_{i,j,c}, \qquad \mu_{i,c}^a = \sum_j \pi_{j|c}^a (1 - R_{i,j,c}), \quad (35)$$

and let $\mathrm{TV}(p,q) = \frac{1}{2}\|p-q\|_1$. Under the calibration design, the means in Eqs. 25–26 equal $1 - R_i^{\mathrm{cal}}$ and $\mu_{i,c}^{\mathrm{cal}}$, respectively.

Theorem V.6 (Effective coverage under mixture shift)**.** Suppose that the within-class client mixtures differ by at most η_c. On the simultaneous calibration event, every adaptively selected set $\widehat{S}$ satisfies

$$\begin{aligned} C_\mu^{\mathrm{cal}}(\widehat{S};\boldsymbol{\vartheta} + \boldsymbol{\eta} + 2\boldsymbol{r}) &\leq C_{\mathrm{cert}}(\widehat{S};\boldsymbol{\vartheta} + \boldsymbol{\eta}) \\ &\leq C_\mu^{\mathrm{dep}}(\widehat{S};\boldsymbol{\vartheta}) \leq C_{\mathrm{sem}}(\widehat{S}). \end{aligned} \quad (36)$$

where $(\boldsymbol{r})_c = r_c$ and $(\boldsymbol{\eta})_c = \eta_c$.

Moreover, if $\mathrm{TV}(\pi^{\mathrm{dep}}, \pi^{\mathrm{cal}}) \leq \eta$, then

$$R_i^{\mathrm{dep}} \leq R_i^{\mathrm{cal}} + \eta. \quad (37)$$

Proof. Theorem V.2 and the mixture assumption give

$$\mu_{i,c}^{\mathrm{cal}} \geq \vartheta_c + \eta_c + 2r_c \Longrightarrow a_{i,c} \geq \vartheta_c + \eta_c \Longrightarrow \mu_{i,c}^{\mathrm{dep}} \geq \vartheta_c.$$

Intersecting these implications with the fixed support edge $\Gamma_{i,c} = 1$, taking the existential quantifier over selected candidates, and summing class weights proves Eq. 36.

For the final claim, total variation bounds the change in the expectation of any $[0,1]$-valued function. Applying this bound to the cell risks yields Eq. 37. □

Thus, C_{cert} connects the historical class support of selected detectors with current class-conditional Brier utility under client-mixture shift. The executed selector uses C_{sem} and Φ; C_{cert} and the mixture-sensitivity bound are derived analysis quantities for the frozen returned set.

VI. Evaluation

In this section, we evaluate the implemented SiloStitch pipeline, including its calibration-gated, budgeted detector-selection and shared-stacker pipeline. We organize the evaluation around five questions that follow the method's decision order:

- **RQ1: Utility under a budget.** Can SiloStitch reduce the retained detector pool and its logical payload while preserving detection utility?
- **RQ2: Semantic priority.** When client label support is heterogeneous, does the semantic objective change the selected set, and what does that change cost?
- **RQ3: Design dependence.** How sensitive are the results to the secondary objective, semantic parameters, and learner architecture?
- **RQ4: Score perturbation.** How does Laplace perturbation affect shared-stacker utility?
- **RQ5: Practicality.** Does the primary solver match exhaustive search on tractable cases, how does it scale, and where do computation and logical communication occur?

We evaluate task utility, source-side support, perturbation sensitivity, and systems cost, connecting the selector's optimization target to end-to-end outcomes.

A. Experimental Setup

Datasets. We use four cybersecurity datasets: CIC-DDoS2019 (DDOS) [23], CICMalDroid2020 (MALDROID) [24], CIC-Darknet2020 (DARKNET) [25], and CIRA-CIC-DoHBrw-2020 (DOH) [26]. Table II reports the frozen data sizes. Each task contains 66 clients and therefore 66 candidate detectors. Within each dataset, all clients use the same numeric class vocabulary. The controlled partitions vary source-support allocation while holding class semantics fixed.

Protocol. We divide each training pool into disjoint 60%, 10%, and 30% partitions for candidate fitting, selection calibration, and shared stacker fitting, respectively. The held-out test partition is opened only after both the selected detector set and stacker are frozen. Unless stated otherwise, we use five protocol seeds, LightGBM [27] for both local detectors and the global stacker (L+L), a 50% logical-byte budget, minimum semantic support $n_{\min} = 2$, inverse-square-root class weights, and equal weights for U, C_{pred}, and D_{rep}. The fixed calibration threshold defines selection eligibility throughout the experiment.

Table II
FROZEN DATASETS USED IN THE EVALUATION. "TRAIN" IS THE POOL SPLIT FURTHER INTO CANDIDATE FITTING, SELECTION CALIBRATION, AND STACKER FITTING; THE TEST SET REMAINS SEALED UNTIL ALL SELECTIONS AND STACKERS ARE FIXED.

Dataset	Clients	Train records	Test records	Features	Classes
DDOS	66	3,355,966	1,127,526	79	14
MALDROID	66	8,118	3,480	470	5
DARKNET	66	99,066	42,459	76	11
DOH	66	1,005,708	431,034	29	4

Operating conditions. We use three contemporaneous operating conditions: (i) *All encoders*, the unbudgeted cost ceiling; (ii) *Random budget*, which packs a random feasible set and averages 20 fixed selection seeds within each protocol seed; and (iii) *Pointwise utility*, which ranks candidates by individual calibration utility. For diagnosis, a matched *no-semantic* ablation uses the same secondary fill-and-swap solver as SILOSTITCH but removes the primary semantic priority. All reported conditions share the candidate pool, data partitions, budget definition, stacker capacity, and sealed test set.

Metrics and statistics. Macro-F1 is our primary task metric because all four datasets are multiclass and imbalanced. We also record accuracy, balanced accuracy, worst shared-class recall, weighted semantic coverage, and per-dataset logical cost. Unless noted otherwise, points and table entries are means with 95% Student-t confidence intervals over five protocol seeds. We pair conditions by frozen protocol seed; pre-specified inferential families use Holm correction. An independent final validator re-computes accuracy, Macro-F1, balanced accuracy, and worst shared-class recall from confusion matrices, rebuilds the exact coverage oracle from candidate evidence, and checks every budget and input identity.

Implementation. We run the experiments as a single-machine federation replay under Linux and Python 3.9.25, using NumPy 1.26.4, scikit-learn 1.3.2, LightGBM 4.6.0, and XGBoost 1.4.2. We report the logical protocol payload defined in Eq. 5 together with local wall-clock time.

B. RQ1: Can a Budgeted Selection Preserve Utility?

Primary comparison. Table III compares the 50%-budget conditions with the unbudgeted all-encoder condition. SILOSTITCH uses approximately half of the all-encoder logical payload on every dataset. Relative to all encoders, its Macro-F1 decreases by 0.0258 on DDOS, 0.0052 on MALDROID, 0.0019 on DARKNET, and 0.0009 on DOH. Thus, the half-cost configuration closely tracks the all-encoder condition on three datasets, while DDOS exhibits a larger utility gap.

Budget response. Table IV follows SILOSTITCH itself at 25%, 50%, and 75% target budgets. MALDROID improves steadily as the budget grows, DDOS and DARKNET exhibit non-monotone responses, and DOH remains stable. Additional encoders therefore deliver dataset-specific returns.

Answer to RQ1. At approximately half the logical payload, SILOSTITCH stays within 0.0052 Macro-F1 of all encoders on MALDROID and within 0.002 on DARKNET and DOH; the DDOS gap is 0.0258. Across all four datasets, it turns the budget into an exact, independently validated semantic primary optimum.

C. RQ2: When Does Semantic Priority Matter?

Semantic priority preserves the maximum attainable source support before secondary utility refinement. DDOS exercises this mechanism through heterogeneous support; in every evaluated seed, its matched secondary solver independently reaches the same maximum and selects the same set. The other three primary datasets already attain maximum support under the matched secondary solution.

Controlled heterogeneity. We next vary label allocation with IID and Dirichlet partitions ($\alpha \in \{10, 1, 0.5, 0.1\}$). Smaller α concentrates labels on fewer clients. Every constructed partition must leave at least two candidate-fit classes at each client. DARKNET remains constructible from IID allocation through $\alpha = 0.1$. MALDROID remains constructible under IID and $\alpha = 10$, with seed-dependent feasibility at $\alpha = 1$. These feasibility boundaries determine the controlled settings evaluated below.

Across the constructible controlled settings, SILOSTITCH and the no-semantic control select identical sets at 25%, 50%, and 75% budgets. Their Jaccard similarity is 1, and their paired differences in semantic coverage, worst shared-class recall, and Macro-F1 are exactly zero.

Answer to RQ2. We compute and independently re-validate the exact maximum semantic coverage in every evaluated primary and controlled condition. Across these settings, SILOSTITCH selects the same set as the matched control, delivering the exact semantic optimum at identical utility and logical cost.

D. RQ3: How Dependent Are the Results on Design Choices?

Secondary components. Figure 2 removes record utility U, classwise predictive evidence C_{pred}, and pool representativeness D_{rep} in turn. The ablation response is dataset-specific. At the 50% budget, removing D_{rep} changes Macro-F1 by $+0.0181$ on DDOS and -0.0094 on DARKNET. The secondary terms therefore guide a coverage-preserving local search whose utility effect adapts to the dataset.

Semantic parameters. On two fixed DARKNET partitions and two architectures, varying $n_{\min} \in \{1, 2, 5, 10\}$ and replacing inverse-square-root class weights with uniform weights leaves the selected set, semantic coverage, Macro-F1, and worst recall unchanged across the tested grid.

Learner architecture. We evaluate all four combinations of XGBoost [28] and LightGBM local/global learners. L+X is

Table III
MACRO-F1 AT THE 50% TARGET BUDGET. VALUES ARE MEANS WITH 95% CONFIDENCE INTERVALS OVER FIVE PROTOCOL SEEDS.

Method	DDOS	MALDROID	DARKNET	DOH
All encoders	0.6412 ± 0.0121	0.8449 ± 0.0123	0.5972 ± 0.0128	0.6073 ± 0.0042
Random budget	0.6079 ± 0.0287	0.8344 ± 0.0100	0.5931 ± 0.0071	0.6074 ± 0.0028
Pointwise utility	0.6303 ± 0.0437	0.8374 ± 0.0132	0.5921 ± 0.0168	0.6069 ± 0.0042
SILOSTITCH	0.6153 ± 0.0453	0.8397 ± 0.0121	0.5953 ± 0.0148	0.6064 ± 0.0038

Table IV
SILOSTITCH MACRO-F1 ACROSS TARGET BUDGETS. VALUES ARE FIVE-SEED MEANS WITH 95% CONFIDENCE INTERVALS.

Target budget	DDOS	MALDROID	DARKNET	DOH
25%	0.6039 ± 0.0712	0.8224 ± 0.0150	0.5830 ± 0.0086	0.6057 ± 0.0061
50%	0.6153 ± 0.0453	0.8397 ± 0.0121	0.5953 ± 0.0148	0.6064 ± 0.0038
75%	0.6104 ± 0.0433	0.8443 ± 0.0090	0.5903 ± 0.0130	0.6077 ± 0.0036

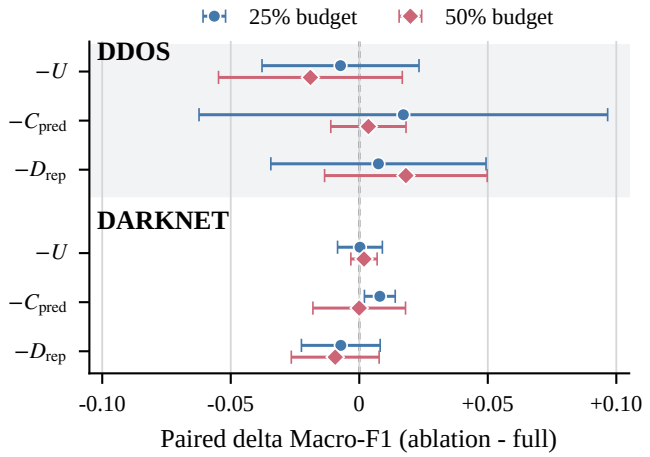

Figure 2. Paired Macro-F1 change after removing one secondary component. Points distinguish the 25% and 50% budgets; horizontal bars are 95% confidence intervals over five protocol seeds.

Table V
SILOSTITCH ACCURACY FOR FOUR LOCAL/GLOBAL LEARNER COMBINATIONS. X AND L DENOTE XGBOOST AND LIGHTGBM, RESPECTIVELY; VALUES ARE MEANS OVER FIVE PROTOCOL SEEDS.

Dataset	X+X	L+X	X+L	L+L
DDOS	0.621	**0.642**	0.641	0.624
MALDROID	0.826	0.854	0.847	**0.860**
DARKNET	0.824	0.826	0.829	**0.832**
DOH	0.801	0.800	**0.807**	**0.807**

best on DDOS, L+L is best on MALDROID and DARKNET, and X+L and L+L tie on DOH. The pipeline supports both learner families, with the strongest backend sensitivity appearing on DDOS.

Answer to RQ3. The selected solution is invariant across the tested semantic-parameter grid, and the pipeline supports all four learner combinations. Component and backend choices provide dataset-specific tuning opportunities.

Table VI
MACRO-F1 SENSITIVITY TO PER-BLOCK LAPLACE PERTURBATION ON DDOS. FIVE-SEED MEANS ARE REPORTED WITH 95% CONFIDENCE INTERVALS.

ϵ	Noise scale	Selected (50%)	All 66
No noise	0	0.6153 ± 0.0453	0.6412 ± 0.0121
50	0.04	0.5861 ± 0.0439	0.6037 ± 0.0078
20	0.10	0.5932 ± 0.0244	0.5999 ± 0.0234
10	0.20	0.5910 ± 0.0117	0.5918 ± 0.0191
7	0.286	0.5939 ± 0.0136	0.5874 ± 0.0174
5	0.40	0.5875 ± 0.0091	0.5777 ± 0.0163
3	0.667	0.5655 ± 0.0120	0.5870 ± 0.0046
1	2.00	0.5105 ± 0.0301	0.5298 ± 0.0400

E. RQ4: What Utility Remains after Laplace Perturbation?

We can perturb every selected score block before logical upload. To isolate utility sensitivity, we freeze the 50%-budget selection first and then add independent Laplace noise with scale $2/\epsilon$ to stacker-fit encodings. We compare this selected configuration with all 66 encoders on DDOS over five protocol seeds, keeping test encodings fixed for evaluation. This centralized replay samples the distribution in Eq. 22; the execution model of Theorem V.1 instead draws fresh private randomness at each client before upload.

Table VI reports Macro-F1 for no noise and $\epsilon \in \{50, 20, 10, 7, 5, 3, 1\}$. Here, ϵ controls the per-block perturbation and the corresponding noise scale is $2/\epsilon$. At the strongest tested perturbation ($\epsilon = 1$), Macro-F1 falls from 0.6153 to 0.5105 for the 50%-budget selection and from 0.6412 to 0.5298 for all encoders. The intermediate means fluctuate as ϵ decreases; the largest observed drop occurs at $\epsilon = 1$.

Answer to RQ4. The seven perturbation levels expose a utility–perturbation profile: intermediate means fluctuate, while $\epsilon = 1$ produces the largest observed utility drop. Together with Theorem V.1, these results connect the Laplace release rule to its measured stacker utility.

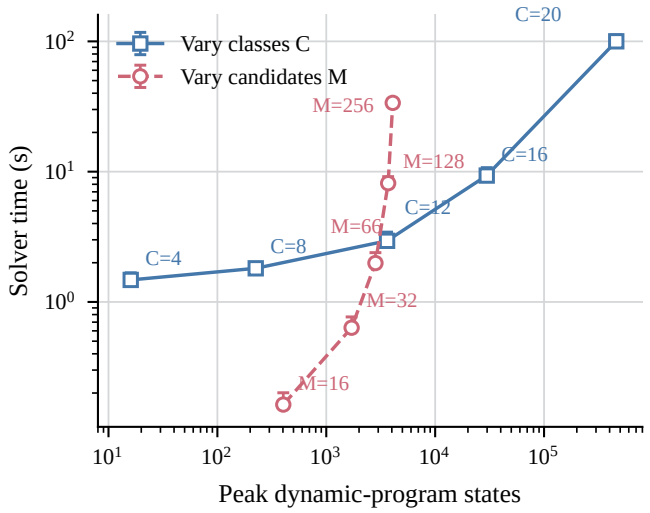

Figure 3. Sparse dynamic-program scaling. The blue and rose trajectories independently vary semantic classes and candidates using separate fixed fixtures. Both axes are logarithmic; whiskers extend from median to 95th-percentile time over ten repeats.

F. RQ5: Is the Selector and Pipeline Practical?

Exactness and scaling. We first compare the bitmask dynamic program with exhaustive lexicographic search on a suite of small oracle instances. The primary coverage value agrees throughout the suite, and repeated runs are deterministic. Figure 3 then separates growth in the number of semantic classes C from growth in the candidate count M. At the main-scale point $M = 66, C = 12$, the isolated solver uses a median of 1.99 seconds, reaches 2,829 states, and peaks at 98.27 MiB RSS. Increasing M to 256 at $C = 12$ raises median time to 33.73 seconds, whereas increasing C to 20 at $M = 66$ reaches 460,405 states, 100.10 seconds, and 580.45 MiB. Thus, semantic dimension, rather than candidate count alone, is the dominant capacity driver.

End-to-end replay cost. Table VII integrates wall-clock and logical-communication evidence. Full group time includes all reported methods and repetitions, whereas the preceding microbenchmark isolates one primary solve. Stacker fitting is the largest measured stage on MALDROID, DARKNET, and DOH; the multi-method selection stage is largest on DDOS. At the 50% budget, SILOSTITCH reduces total logical payload by approximately half on each dataset.

Answer to RQ5. The solver exactly matches exhaustive search throughout the oracle suite and completes the 66-candidate benchmark in 1.99 seconds. Scaling isolates semantic dimension as the dominant capacity driver, while the end-to-end replay confirms an approximately 50% reduction in logical payload.

VII. DISCUSSION

The lexicographic objective answers two distinct deployment questions. Weighted semantic coverage specifies which source-side class support should remain whenever the budget permits; the secondary score distinguishes current prediction behavior only after the best attainable coverage has been fixed. An instance-specific scalarization can reproduce this order when its minimum positive coverage gap is known, but no universal finite coefficient works across arbitrary class weights and candidate pools. The ordered formulation therefore makes the intended priority explicit. Semantic coverage measures preserved source-side breadth, while class-conditional Brier lower bounds and sealed-test metrics measure current competence. The effective-coverage certificate connects these two views by requiring a selected, source-supported detector to exceed a deployment-specific class-utility threshold.

The registered profile sets $\tau_L = 1$, so the upper-confidence gate acts as a finite-sample admissibility screen over normalized Brier risk. The simultaneous calibration event remains valid after adaptive selection. Within the resulting pool, the bitmask dynamic program computes the exact primary value under its registered capacity guard, and the subsequent fill-and-swap stages preserve that value while providing a one-swap local certificate.

Client heterogeneity determines the population described by the calibration statistics. The implementation targets the record-weighted client–class mixture. Under stable cell-conditional risks and a deployment mixture within total variation η of the calibration mixture, deployment Brier risk is at most calibration risk plus η. The logical-byte budget then provides a reproducible accounting unit for model delivery and aligned score blocks across all configurations.

The privacy protocol asks each client to perturb selected score blocks with fresh private Laplace randomness before upload. The theorem assigns ϵ_b to one block and composes to $s\epsilon_b$ for s selected blocks of one record. Our centralized sensitivity profile samples the same post-selection distribution and measures its utility over seven perturbation levels. A distributed realization places this randomization at each client as specified by the protocol.

VIII. CONCLUSION

In summary, SILOSTITCH integrates pretrained intrusion detectors under a hard logical-byte budget without changing their parameters. It aligns their outputs, screens candidates with held-out calibration data, maximizes weighted semantic coverage exactly under an explicit capacity guard, and refines current utility and representativeness while preserving that value. Across four datasets, the 50% target budget cuts logical payload by 49.3–50.4% and closely tracks all-encoder Macro-F1 on MALDROID, DARKNET, and DOH. The selected detectors feed one shared stacker, and client-side Laplace randomization gives label-conditional local privacy to uploaded score blocks. Together, the calibration, solver, privacy, and client-mixture results make post-training detector integration an auditable deployment decision.

REFERENCES

[1] Australian Signals Directorate's Australian Cyber Security Centre, "Best practices for event logging and threat detection," Cyber.gov.au, 2024. [Online]. Available: https://www.cyber.gov.au/business-government/detecting-responding-to-threats/event-logging/best-practices-for-event-logging-and-threat-detection

Table VII
SINGLE-MACHINE REPLAY TIME AND LOGICAL PAYLOAD. TIMES ARE MEANS OVER FIVE PROTOCOL SEEDS. "DOMINANT" IS THE LARGEST RECORDED STAGE; LOGICAL REDUCTION COMPARES SILOSTITCH WITH ALL ENCODERS.

Dataset	Total group (s)	Dominant stage	Stage time (s)	All (MiB)	SILOSTITCH (MiB)	Reduction
DDOS	3,005.16	Selection	1,026.34	7,040.05	3,489.07	50.4%
MALDROID	47.03	Stacker fit	35.21	662.66	335.94	49.3%
DARKNET	2,550.50	Stacker fit	2,272.66	3,436.78	1,732.29	49.6%
DOH	512.34	Stacker fit	170.04	2,942.76	1,488.51	49.4%

[2] Microsoft, "Microsoft Entra data retention," Microsoft Learn, 2026, accessed: Aug. 10, 2026. [Online]. Available: https://learn.microsoft.com/en-us/entra/identity/monitoring-health/reference-reports-data-retention

[3] E. Tabassi, "Artificial intelligence risk management framework (AI RMF 1.0)," National Institute of Standards and Technology, Tech. Rep. NIST AI 100-1, 2023.

[4] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[5] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of Machine Learning and Systems*, I. Dhillon, D. Papailiopoulos, and V. Sze, Eds., vol. 2. MLSys, 2020, pp. 429–450. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2020/hash/1f5fe83998a09396ebe6477d9475ba0c-Abstract.html

[6] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 6357–6368. [Online]. Available: https://proceedings.mlr.press/v139/li21h.html

[7] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[8] V. Pourahmadi, H. A. Alameddine, M. A. Salahuddin, and R. Boutaba, "Spotting anomalies at the edge: Outlier exposure-based cross-silo federated learning for DDoS detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 4002–4015, 2023.

[9] J. Xu, C. Li, Y. Zheng, and Z. Li, "ENTENTE: Cross-silo intrusion detection on network log graphs with federated learning," in *Network and Distributed System Security Symposium*, 2026. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2026-s93-paper.pdf

[10] Y. Qing, Q. Yin, X. Deng, Y. Chen, Z. Liu, K. Sun, K. Xu, J. Zhang, and Q. Li, "Low-quality training data only? a robust framework for detecting encrypted malicious network traffic," in *Network and Distributed System Security Symposium*, 2024. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2024-81-paper.pdf

[11] R. Xie, J. Cao, E. Dong, M. Xu, K. Sun, Q. Li, L. Shen, and M. Zhang, "Rosetta: Enabling robust TLS encrypted traffic classification in diverse network environments with TCP-Aware traffic augmentation," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, 2023, pp. 625–642. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/xie

[12] S. Yang, X. Zheng, J. Li, J. Xu, and E. C. H. Ngai, "CoLD: Collaborative label denoising framework for network intrusion detection," in *Network and Distributed System Security Symposium*, 2026. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2026-s1950-paper.pdf

[13] Q. Dong, A. Muhammad, F. Zhou, C. Xie, T. Hu, Y. Yang, S.-H. Bae, and Z. Li, "ZooD: Exploiting model zoo for out-of-distribution generalization," in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/cd305fdee96836d5cc1de94577d71b61-Abstract-Conference.html

[14] Y. Wei, Z. Hu, L. Shen, Z. Wang, Y. Li, C. Yuan, and D. Tao, "Task groupings regularization: Data-free meta-learning with heterogeneous pre-trained models," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 52 573–52 587. [Online]. Available: https://proceedings.mlr.press/v235/wei24e.html

[15] X. Zhao, G. Sun, R. Cai, Y. Zhou, P. Li, P. Wang, B. Tan, Y. He, L. Chen, Y. Liang, B. Chen, B. Yuan, H. Wang, A. Li, Z. Wang, and T. Chen, "Model-GLUE: Democratized LLM scaling for a large model zoo in the wild," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://openreview.net/forum?id=M5JW7O9vc7

[16] S. Thirumuruganathan, M. Nabeel, E. Choo, I. Khalil, and T. Yu, "SIRAJ: A unified framework for aggregation of malicious entity detectors," in *43rd IEEE Symposium on Security and Privacy*. IEEE, 2022, pp. 507–521.

[17] W. Qiu, Y. Feng, Y. Li, Y. Chang, K. Qian, B. Hu, Y. Yamamoto, and B. W. Schuller, "Fed-MStacking: Heterogeneous federated learning with stacking misaligned labels for abnormal heart sound detection," *IEEE Journal of Biomedical and Health Informatics*, vol. 28, no. 9, pp. 5055–5066, Sep. 2024.

[18] Y. Birman, S. Hindi, G. Katz, and A. Shabtai, "Cost-effective ensemble models selection using deep reinforcement learning," *Information Fusion*, vol. 77, pp. 133–148, 2022.

[19] T. Qi, F. Wu, C. Wu, L. He, Y. Huang, and X. Xie, "Differentially private knowledge transfer for federated learning," *Nature Communications*, vol. 14, p. 3785, 2023. [Online]. Available: https://www.nature.com/articles/s41467-023-38794-x

[20] S. Maddock, G. Cormode, T. Wang, C. Maple, and S. Jha, "Federated boosted decision trees with differential privacy," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 2249–2263. [Online]. Available: https://doi.org/10.1145/3548606.3560687

[21] J. C. Duchi, M. I. Jordan, and M. J. Wainwright, "Local privacy and statistical minimax rates," in *2013 IEEE 54th Annual Symposium on Foundations of Computer Science*, 2013, pp. 429–438.

[22] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of cryptography conference*. Springer, 2006, pp. 265–284.

[23] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (ddos) attack dataset and taxonomy," in *2019 International Carnahan Conference on Security Technology (ICCST)*. IEEE, 2019, pp. 1–8.

[24] S. Mahdavifar, A. F. A. Kadir, R. Fatemi, D. Alhadidi, and A. A. Ghorbani, "Dynamic android malware category classification using semi-supervised deep learning," in *IEEE International Conference on Dependable, Autonomic and Secure Computing*. IEEE, 2020, pp. 515–522.

[25] A. Habibi Lashkari, G. Kaur, and A. Rahali, "Didarknet: A contemporary approach to detect and characterize the darknet traffic using deep image learning," in *2020 the 10th International Conference on Communication and Network Security*, 2020, pp. 1–13.

[26] M. MontazeriShatoori, L. Davidson, G. Kaur, and A. H. Lashkari, "Detection of doh tunnels using time-series classification of encrypted traffic," in *2020 IEEE International Conference on Dependable, Autonomic and Secure Computing*. IEEE, 2020, pp. 63–70.

[27] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," *Advances in neural information processing systems*, vol. 30, pp. 3146–3154, 2017.

[28] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.